DATA STRUCTURES

Arrays ADT

Arrays

- An array is defined as
  - Ordered collection of a fixed number of elements
  - All elements are of the same data type

- Basic operations
  - Direct access to each element in the array
  - Values can be retrieved or stored in each element

Properties of an Array

- **Ordered**
  - Every element has a well-defined position
  - First element, second element, etc.
- **Fixed size or capacity**
  - Total number of elements are fixed
- **Homogeneous**
  - Elements must be of the same data type (and size)
  - Use arrays only for homogeneous data sets
- **Direct access**
  - Elements are accessed directly by their position
  - Time to access each element is same
  - Different to sequential access where an element is only accessed after the preceding elements

Declaring Arrays in C/C++

```
datatype arrayName[intExp];
```

- datatype – Any data type, e.g., integer, character, etc.
- arrayName – Name of array using any valid identifier
- intExp – Constant expression that evaluates to a positive integer

```
const int SIZE =10;
int list[SIZE];
```

- Compiler reserves a block of consecutive memory locations enough to hold SIZE values of type int

Accessing Arrays in C/C++

arrayName[indexExp];

- indexExp – called index, is any expression that evaluates to a positive integer
- In C/C++
  - Array index starts at 0
  - Elements of array are indexed 0, 1, 2, …, SIZE-1
  - [ ] is called array subscripting operator

```
int value =list[2]
list[0]=value*2;
```

C/C++ Implementation of an Array ADT

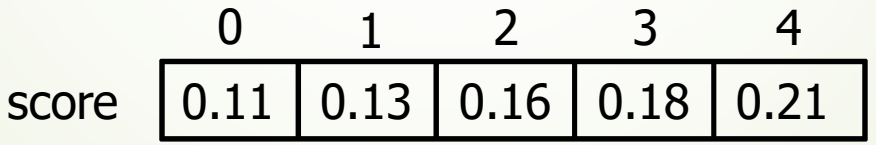
As an ADT	In C/C++
Ordered	Index: 0,1,2, …SIZE-1
Fixed Size	intExp is constant
Homogeneous	dataType is the type of all elements
Direct Access	Array subscripting operator [ ]

Array Initialization in C/C++

dataType arrayName[intExp]={listofvalues}

- In C/C++, arrays can be initialized at declaration
  - `intExp` is optional: Not necessary to specify the size

- Example: Numeric arrays
  - `double score[] ={0.11, 0.13,0.16,0.18,0.21}`



Array Initialization in C/C++

- Fewer values are specified than the declared size of an array
  - Numeric arrays: Remaining elements are assigned zero
  - Character arrays: Remaining elements contains null character '\0' ASCII code of '\0' is zero

double score[5] = {0.11, 0.13, 0.16}

	0	1	2	3	4
score	0.11	0.13	0.16	0	0

char name[6] = {'J', 'O', 'H', 'N'}

	0	1	2	3	4	5
name	J	O	H	N	\0	\0

- If more values are specified than declared size of an array
  - Error is occurred: Handling depends on compiler

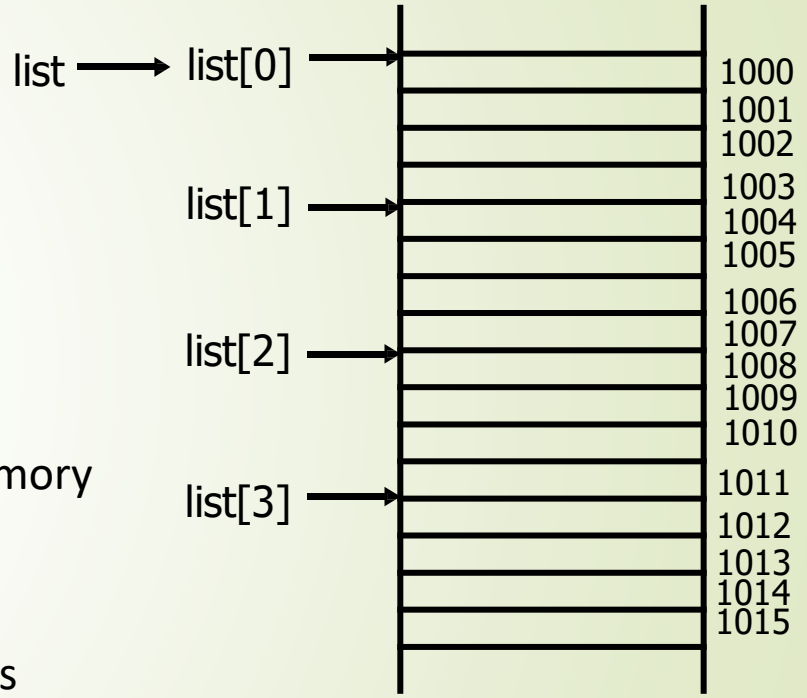
Array Addressing

- Consider an array declaration: i n t l i s t [4] ={ 1 , 2 , 4 , 5 }
  - Compiler allocates a block of four memory spaces
  - Each memory space is large enough to store an i n t value
  - Four memory spaces are contiguous
- Base address
  - Address of the first byte (or word) in the contiguous block of memory
  - Address of the memory location of the first array element
    - Address of element list[0]
- Memory address associated with array_Name stores the base address

cout << list << endl;

cout << *list << endl;

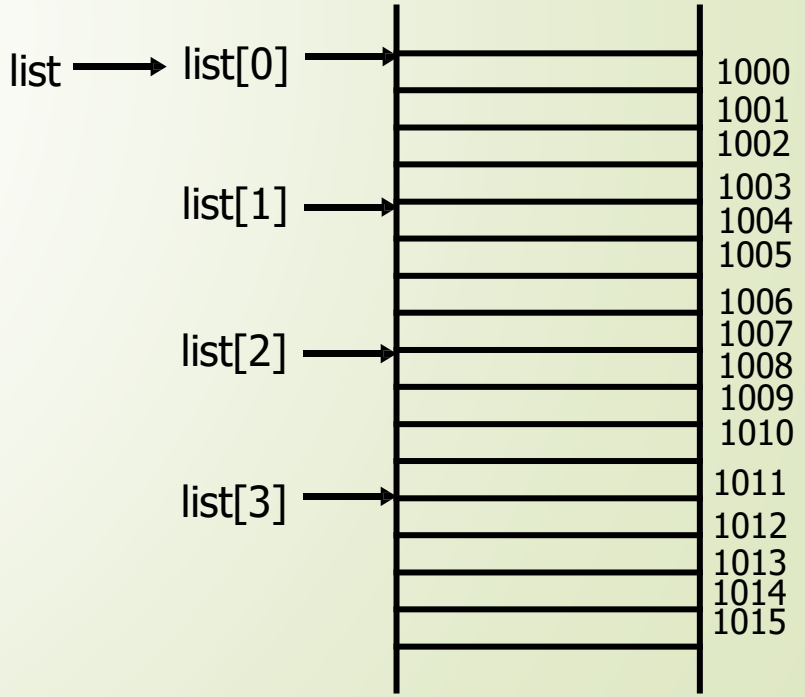
- * is dereferencing operator Returns content of a memory location



Array Addressing

- Consider a statement: cout<<list[3];
- Requires array reference list[3]be translated into memory address
  - Offset: Determines the address of a particular element w.r.t. base address
- Translation
  - Base address + offset = 1000 + 3 x sizeof(int)=1012
  - Content of address 1012 are retrieved & displayed
- An address translation is carried out each time an array element is accessed
- What will be printed and why?

cout<<*(list+3)<<endl;



Multidimensional Arrays

- Most languages support arrays with more than one dimension
  - High dimensions capture characteristics/correlations associated with data

- Example: A table of test scores for different students on several tests
  - 2D array is suitable for storage and processing of data

	Test 1	Test 2	Test 3	Test 4
Student 1	99.0	93.5	89.0	91.0
Student 2	66.0	68.0	84.5	82.0
Student 3	88.5	78.5	70.0	65.0
:	:	:	:	:
:	:	:	:	:
Student N	100.0	99.5	100.0	99.0

Two Dimensional Arrays – Declaration

dataType arrayName[intExp1][intExp2];

- intExp1 – constant expression specifying number of rows
- intExp2 – constant expression specifying number of columns
- Example:

```
const int   NUM_ROW =2, NUM_COLUMN =    4;
double scoreTable [NUM_ROW][NUM_COLUMN];
```

- Initialization:

```
Double scoreTable [ ] [ 4 ]     = {{0.5,  0.6,0.3},
                                   {0.6,0.3,0.8}};
```

  - List the initial values in braces, row by row
  - May use internal braces for each row to improve readability

Two Dimensional Arrays – Processing

arrayName[indexExp1][indexExp2];

- indexExp1 – row index
- indexExp2 – column index
- Rows and columns are numbered from 0
- Use nested loops to vary two indices
- Row-wise or column-wise manner

	Test 1	Test 2	Test 3	Test 4
Student 1	99.0	93.5	89.0	91.0
Student 2	66.0	68.0	84.5	82.0
Student 3	88.5	78.5	70.0	65.0
:	:	:	:	:
:	:	:	:	:
Student N	100.0	99.5	100.0	99.0

Higher Dimensional Arrays

- Example: Store and process a table of test scores
  - For several different students
  - On several different tests
  - Belonging to different semesters

const int SEMS =10, STUDENTS = 30, TESTS = 4;

double ThreeDimArray[SEMS][STUDENTS][TESTS];

- What is represented by gradebook[4][2][3]?
  - Score of 3rd student belonging to 5th semester on 4th test

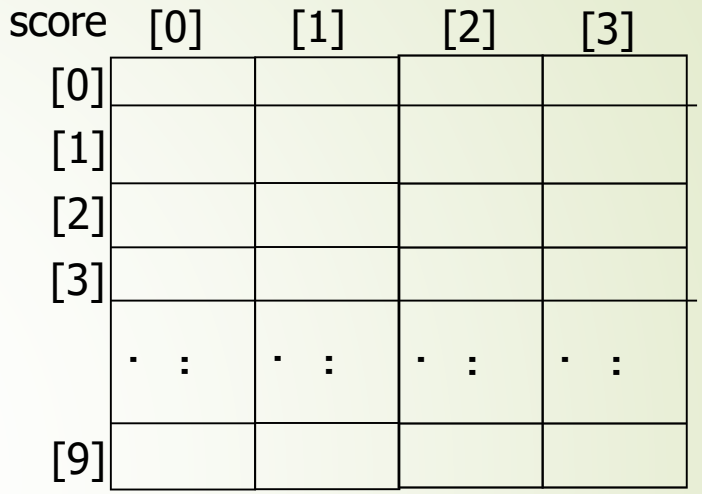
```
int arr[3][3][3]=
        { {{10,20,30},{40,50,60},{70,80,90}},    //elements of block 1
          {{11,22,33},{44,55,66},{77,88,99}},    //elements of block 2
          {{12,23,34},{45,56,67},{78,89,90}}     //elements of block 3
        };

block(1)  10 20 30      block(2)  11 22 33      block(3)  12 23 34
          40 50 60                44 55 66                45 56 67
          70 80 90                77 88 99                78 89 90
                3x3                     3x3                     3x3
```

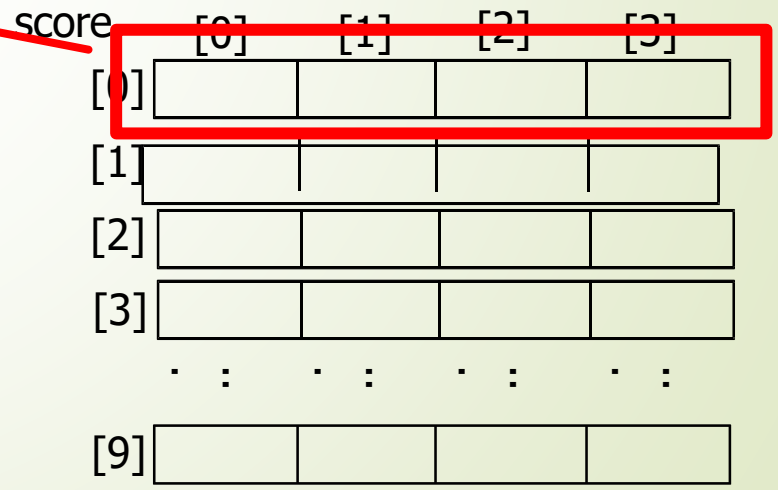
Array of Arrays

- Consider the declaration
  - double score[10][4];
- Another way of declaration
  - One-dimensional (1D) array of rows

```
double    RowOfTable[4];
RowOfTable    score[10];
```

- In detail
  - Declare score as 1D array containing 10 elements
  - Each of 10 elements is 1D array of 4 real numbers (i.e., double)





Array of Arrays

- score[i]
  - Indicates i^{th} row of the table

- score[i][j]
  - Can be thought of as (score[i])[j]
  - Indicates j^{th} element of score[i]

- **An n-dimensional array can be viewed (recursively) as a 1D array whose elements are (n-1)-dimensional arrays**

Implementing Multidimensional Arrays

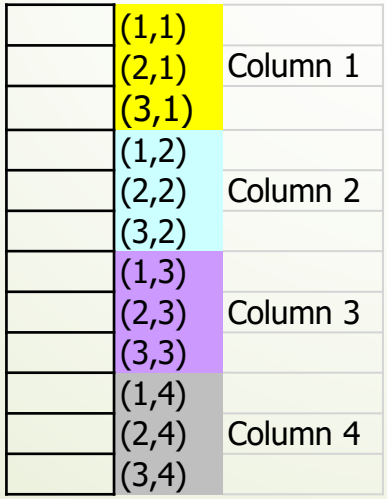
- More complicated than one dimensional arrays
- Memory is organized as a sequence of memory locations
  - One-dimensional (1D) organization
- How to use a 1D organization to store multidimensional data?
- Example:

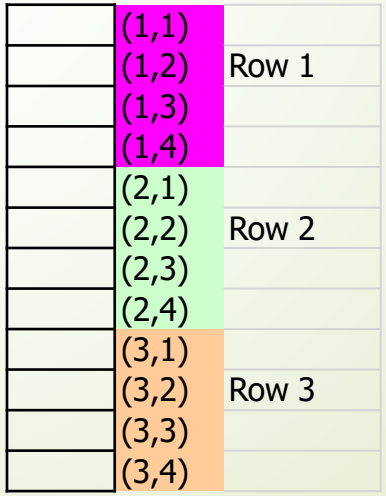
A	B	C	D
E	F	G	H
I	J	K	L

- A character requires single byte
- Compiler request to reserve 12 consecutive bytes
- Two way to store consecutively, i.e., row-wise and column-wise

Two-dimensional Arrays in Memory

- Two ways to be represented in memory
- Column majored
  - Column by column
- Row majored
  - Row by row
- Representation depends upon the programming language





```cpp
#include <iostream>
using namespace std;

// function declaration:
double getAverage(int arr[], int size);

int main () {
   // an int array with 5 elements.
   int balance[5] = {1000, 2, 3, 17, 50};
   double avg;

   // pass pointer to the array as an argument.
   avg = getAverage( balance, 5 ) ;

   // output the returned value
   cout << "Average value is: " << avg << endl;

   return 0;
}
```

```cpp
#include<iostream>
using namespace std;

void processArr(int a[][2]) {
   cout << "element at index 1,1 is " << a[1][1];
}
int main() {
   int arr[2][2];
   arr[0][0] = 0;
   arr[0][1] = 1;
   arr[1][0] = 2;
   arr[1][1] = 3;

   processArr(arr);
   return 0;
}
```